MODELARE ȘI SIMULARE

Săptămâna 4. Teoria cozilor și performanța sistemelor de calcul

Suport de curs · Anul III, Calculatoare și Tehnologia Informației

Ș.l. dr. ing. Cătălin Iapă

Obiective

1. să descrieți un sistem de servire în notația Kendall (A/B/c/K);
2. să calculați indicii de performanță: ρ, L, Lq, W, Wq, throughput;
3. să aplicați legea lui Little și legile operaționale pentru a găsi gâtul de sticlă;
4. să folosiți formulele M/M/1, M/M/c, M/M/1/K, Erlang B și Pollaczek–Khinchine;
5. să explicați de ce latența explodează când utilizarea se apropie de 100%;
6. să analizați o rețea de cozi și să dimensionați un sistem (servere web, cloud, servirea LLM).

Cuvinte cheie: *sistem de servire, șir de așteptare, notația Kendall, proces Poisson, grad de utilizare, throughput, legea lui Little, legi operaționale, gât de sticlă, M/M/1, M/M/c, Erlang B, Erlang C, M/M/1/K, Pollaczek–Khinchine, Kingman, rețea Jackson, dimensionare, latență de coadă, TTFT*

1. Sisteme de servire și legea lui Little

1.1. Sistem de servire, notația Kendall

Un **sistem de servire** (Holban, cap. 3.1) este format din **clienți**, care emit cereri de servire, și din **stații de servire**, fiecare servind un singur client la un moment dat. Clienții care găsesc stațiile ocupate formează un **șir de așteptare**, ordonat după o **disciplină** (FIFO, LIFO, priorități, round-robin, cel mai scurt job întâi). Clientul poate fi o persoană, un pachet, un job, o interogare SQL sau un prompt; stația poate fi un casier, un procesor, un fir de execuție, o conexiune la baza de date sau un GPU. Cozile apar peste tot unde o resursă este partajată, iar întrebarea centrală este cât așteaptă clienții și câte resurse sunt necesare ca așteptarea să fie acceptabilă.

Notația Kendall (1953) descrie un sistem prin A/B/c/K/N/D: A și B sunt distribuțiile timpilor dintre sosiri și ale timpilor de servire (**M** – exponențial, fără memorie; **D** – constant; **G** – oarecare), c este numărul de servere, K capacitatea sistemului, N mărimea populației, iar D disciplina (ultimele trei lipsesc dacă sunt ∞, ∞ și FIFO). De exemplu, benzinăria din studiul de caz al profesorului Holban (sosiri exponențiale cu media 240 u.t., alimentare normală cu media 96 u.t., două pompe) este un sistem M/G/2 cu ρ = 0,2.

Tabelul 1. Modelele folosite în această săptămână

Model	Ce înseamnă	Unde apare
M/M/1	un server, sosiri Poisson, servire exponențială	un CPU, un disc
M/M/c	c servere, o coadă comună	call center, fire de execuție, GPU-uri
M/M/1/K	capacitate finită, clienții în plus sunt refuzați	buffere, cozi limitate (HTTP 429)
M/M/c/c	fără coadă, cereri pierdute	linii telefonice, licențe
M/D/1, M/G/1	servire constantă / oarecare	pachete, cereri variate
G/G/c	totul oarecare	aproape orice sistem real → simulare

1.2. Sosiri Poisson și indicii de performanță

Sosirile sunt modelate de obicei ca **proces Poisson** (Holban, cap. 3.3–3.5): dacă sosirile sunt independente, cu rată constantă λ și fără sosiri simultane, numărul de sosiri într-un interval T are distribuția Poisson(λT), iar timpii dintre sosiri sunt exponențiali cu media 1/λ. Profesorul Holban deduce formula din binomială, împărțind intervalul în bucăți foarte mici. Într-o simulare cu 4 sosiri/s, pe 1 000 de secunde, numărul de sosiri pe secundă a avut media 4,04 și dispersia 3,92, iar histograma a urmat distribuția Poisson (Figura 1). Poisson este un punct de plecare bun când sosirile vin de la multe surse mici și independente, dar nu pentru rafale, joburi sincronizate sau trafic care variază în timpul zilei.

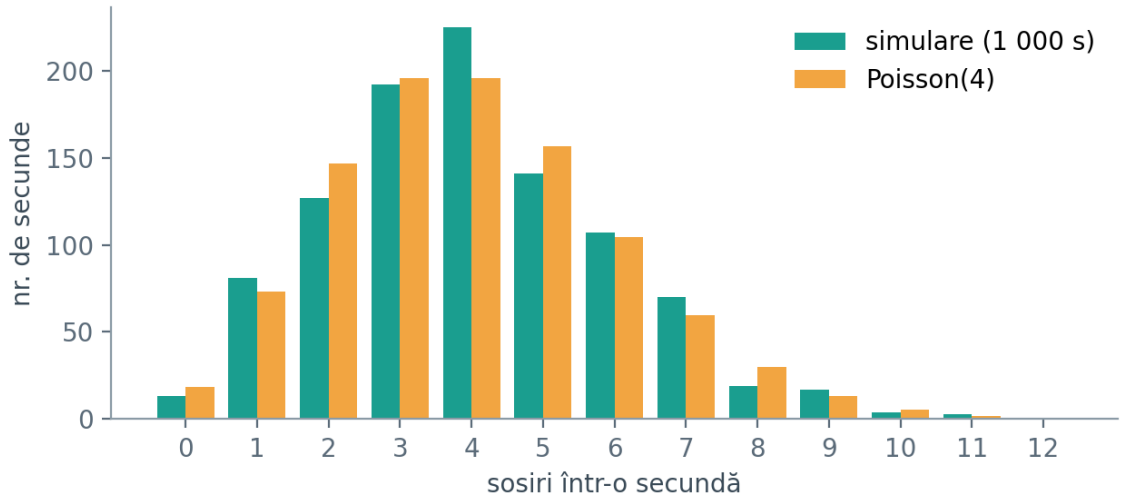


Figura 1. Numărul de sosiri pe secundă pentru un proces Poisson cu 4 sosiri/s (simulare pe 1 000 s) comparat cu distribuția Poisson(4)

Indicii de performanță (Holban, cap. 3.2) sunt: **rata de sosire** λ; **timpul mediu de servire** s și **rata de servire** μ = 1/s; **gradul de utilizare** ρ = λ/(cμ), fracțiunea din timp în care stațiile sunt ocupate; **capacitatea de trecere** (throughput) X = min(λ, cμ); **timpul de așteptare** Wq și **timpul de răspuns** W = Wq + s; **lungimea cozii** Lq și **numărul de clienți din sistem** L. În exemplul lui Holban, 4 clienți sosesc în 10 u.t., cu timpi de servire 0,3; 1,2; 0,7 și 1: λ = 0,4, s = 0,8, μ = 1,25, iar ρ = 3,2/10 = 0,32.

Condiția de **stabilitate** este ρ < 1. Dacă ρ > 1, coada crește liniar în timp, cu aproximativ (λ − cμ)·t; la ρ = 1, coada fluctuează fără margine, ca o plimbare aleatoare (Figura 2). Stabil nu înseamnă însă rapid: throughput-ul și utilizarea nu arată congestia, iar un sistem aproape saturat are un throughput excelent și o latență foarte mare. Operatorul vrea utilizare mare, utilizatorul vrea latență mică – teoria cozilor măsoară exact acest compromis.

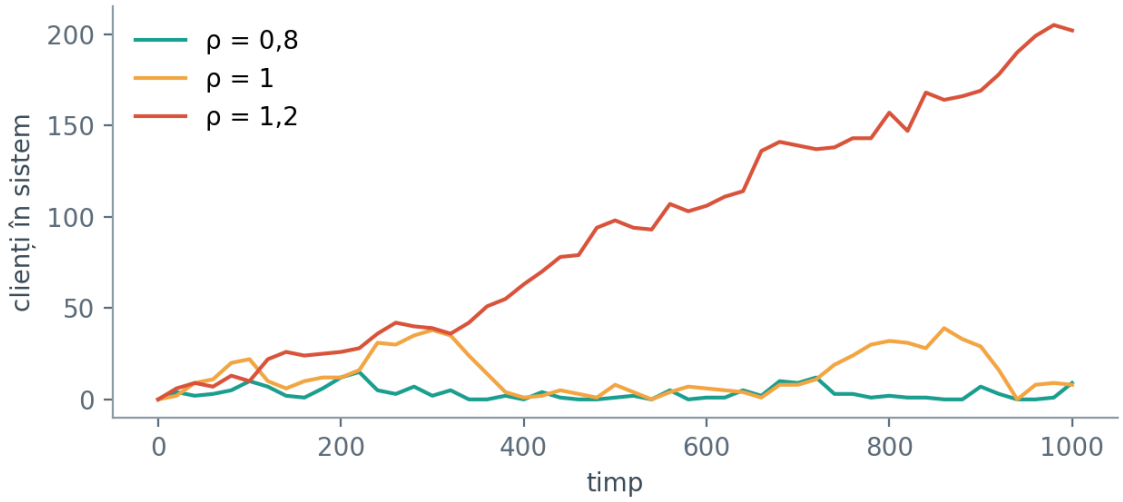


Figura 2. Numărul de clienți dintr-o coadă M/M/1 (μ = 1) pentru trei încărcări, cu aceleași numere aleatoare

1.3. Legea lui Little și legile operaționale

$$L = \lambda \cdot W$$

Legea lui Little (1961): numărul mediu de clienți dintr-un sistem este rata medie de sosire înmulțită cu timpul mediu petrecut în sistem. Demonstrația intuitivă (Gibbens, Cambridge) folosește curbele cumulate ale sosirilor α(t) și plecărilor δ(t): aria dintre ele este, simultan, suma timpilor petrecuți de toți clienții și integrala numărului de clienți în timp (Figura 3). Legea nu presupune nimic despre distribuții, disciplină sau numărul de servere și se aplică oricărei „cutii": cozii ($L_q = \lambda W_q$), serverelor (clienți în servire = $\lambda s = c\rho$) sau unui întreg centru de date. Exemple: un server web cu 200 de cereri/s și 50 ms are în medie 10 cereri în curs; un serviciu LLM cu 20 de cereri/s și 8 s pe generare are 160 de generări simultane. O simulare G/G/1 cu servire lognormală a dat $\hat{\lambda}\cdot\hat{W} = 3{,}114$ și un număr mediu observat de clienți $\hat{L} = 3{,}115$.

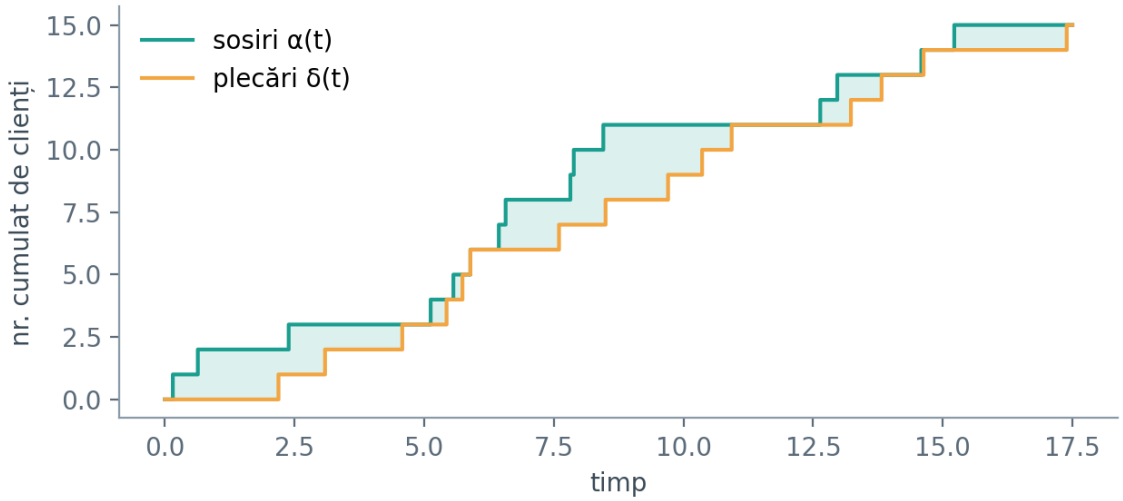


Figura 3. Sosirile și plecările cumulate pentru primii 15 clienți ai unei cozi M/M/1; aria dintre curbe este suma timpilor petrecuți în sistem

Legile operaționale (Jain; Lazowska et al.) folosesc doar mărimi măsurate: legea utilizării $U = X\cdot S$; legea fluxului forțat $X_i = V_i\cdot X$ (o cerere vizitează resursa i de V_i ori); **cererea de servire** $D_i = V_i\cdot S_i$. Resursa cu D_i maxim este **gâtul de sticlă**: throughput-ul sistemului nu poate depăși 1/Dmax, iar timpul de răspuns nu poate coborî

sub ΣD_i. De exemplu, o cerere web care folosește CPU-ul o dată (10 ms), discul de 3 ori (8 ms) și baza de date de 2 ori (15 ms) are D = 10, 24 și 30 ms: baza de date este gâtul de sticlă, X ≤ 33,3 cereri/s și W ≥ 64 ms.

1.4. Simularea unei cozi și prețul variabilității

O coadă FIFO cu un server se simulează cu **recurența lui Lindley**: Wq(i) = max(0, Wq(i − 1) + S(i − 1) − A(i)), unde A(i) este intervalul dintre sosirile clienților i − 1 și i. Pentru M/M/1 cu λ = 0,8 și μ = 1, 200 000 de clienți au dat Wq = 4,11 (teoretic 4), iar 19,7% dintre clienți nu au așteptat deloc (teoretic 1 − ρ = 20%). Așteptările vin în valuri, iar clienții consecutivi au așteptări corelate.

```
rng = np.random.default_rng(5)
n = 200_000
A = rng.exponential(1 / 0.8, n)    # timpi între sosiri, λ = 0,8
S = rng.exponential(1.0, n)        # timpi de servire, μ = 1
Wq = np.zeros(n)
for i in range(1, n):
    Wq[i] = max(0, Wq[i-1] + S[i-1] - A[i])
print(Wq.mean())                   # ≈ 4
```

Variabilitatea creează așteptare. La aceeași utilizare ρ = 0,8 și același timp de servire: D/D/1 (totul regulat) dă Wq = 0; M/D/1 (sosiri aleatoare, servire constantă) dă Wq ≈ 2,0 (teoretic 2); M/M/1 dă Wq ≈ 3,9 (teoretic 4). Așteptarea poate fi redusă deci nu doar prin capacitate, ci și prin uniformizarea sosirilor și a cererilor.

De reținut

Notația Kendall A/B/c/K. Indici: λ, μ, ρ = λ/(cμ), X, W = Wq + s, L, Lq. Stabilitate: ρ < 1 (stabil nu înseamnă rapid). Legea lui Little L = λW, pentru orice sistem. Gâtul de sticlă: Dmax = max $V_i \cdot S_i$; X ≤ 1/Dmax. Variabilitatea singură creează așteptare.

2. Modele clasice de cozi

2.1. M/M/1

O coadă M/M/1 este un **proces naștere–moarte**: un lanț Markov în timp continuu în care starea k (numărul de clienți) crește la o sosire (rata λ) și scade la o plecare (rata μ). Din echilibrul fluxurilor dintre stări vecine, $\pi_k\lambda = \pi_{k+1}\mu$, rezultă $\pi_{k+1} = \rho\pi_k$, iar normarea dă $\pi_0 = 1 - \rho$:

$$\pi_k = (1-\rho)\rho^k, \quad P(N \geq n) = \rho^n, \quad L = \rho/(1-\rho), \quad W = 1/(\mu-\lambda)$$

De asemenea, Lq = ρ²/(1 − ρ) și Wq = ρ/(μ − λ). Timpul de răspuns depinde de **capacitatea rămasă liberă** μ − λ. Măsurat în timpi de servire, W/s = 1/(1 − ρ): ×2 la ρ = 0,5, ×5 la 0,8, ×10 la 0,9, ×20 la 0,95 – curba de tip „crosă de hochei” din Figura 4. Aceeași creștere a încărcării cu 0,1 mărește W cu 13% la ρ = 0,1, dar cu 100% la ρ = 0,8 (Gibbens). Simularea confirmă formula, dar converge greu aproape de ρ = 1, din cauza corelației dintre clienți: la ρ = 0,95 am obținut 19,23 față de 20.

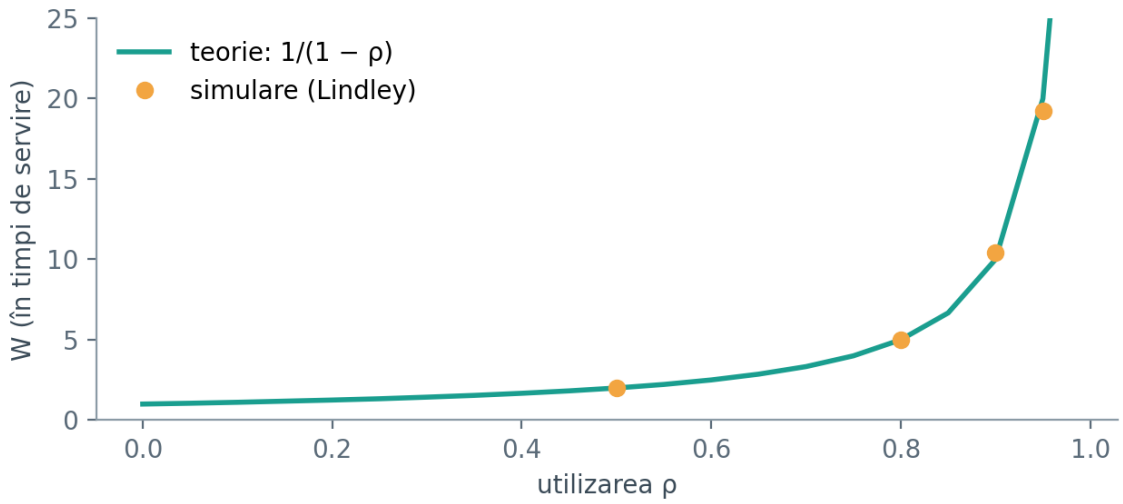


Figura 4. Timpul de răspuns M/M/1 în timpi de servire: formula 1/(1 − ρ) și rezultatele simulării cu 200 000 de clienți

Exemplul frizeriei (Cambridge): λ = 5 clienți/oră, 10 min pe tunsoare (μ = 6/oră). Atunci ρ = 5/6, L = 5, Lq ≈ 4,17, frizerul este liber 1/6 din timp, un client stă în medie 60 min, din care 50 min așteaptă, iar cu 4 scaune un client stă în picioare cu probabilitatea $P(N \geq 5) = \rho^5 \approx 0{,}402$.

Percentilele. În M/M/1, timpul de răspuns are distribuția Exp(μ − λ), deci percentila q este $W(q) = -\ln(1 - q)/(\mu - \lambda)$: mediana ≈ 0,69·W, p95 ≈ 3·W, p99 ≈ 4,6·W. La ρ = 0,8 și s = 1: media 5, mediana 3,47, p95 15,0, p99 23,0. Contractele de nivel de serviciu (SLA) se formulează pe percentile.

2.2. M/M/c și avantajul cozii comune

În M/M/c, c servere identice (rata μ fiecare) servesc o singură coadă; rata de plecare în starea k este min(k, c)·μ, iar sistemul e stabil dacă $\rho = \lambda/(c\mu) < 1$. Mărimea $a = \lambda/\mu$ este **încărcarea oferită**, în erlangi. Probabilitatea ca un client să aștepte este dată de **formula Erlang C**, C(c, a), iar $W_q = C(c, a)/(c\mu - \lambda)$. Numeric, C se calculează din Erlang B: $C = B/(1 - \rho + \rho B)$.

Tabelul 2. Aceeași capacitate totală (2) și aceeași utilizare (ρ = 0,8), λ = 1,6

Configurație	W	Wq
M/M/1, un server cu μ = 2	2,50	2,00
M/M/2, două servere cu μ = 1, coadă comună	2,78	1,78
2 × M/M/1, cozi separate	5,00	4,00

Tabelul 2 și Figura 5 arată lecția **pooling**: o coadă comună pentru mai multe servere este mult mai bună decât aceleași servere cu cozi separate (timpul de răspuns se înjumătățește), pentru că nu mai există situația în care un client așteaptă în timp ce alt server e liber. Un server rapid are cel mai mic W la încărcare mică, pentru că și servirea este mai scurtă; mai multe servere aduc robustețe. Există și o economie de scară: la aceeași utilizare, sistemele mari așteaptă mult mai puțin.

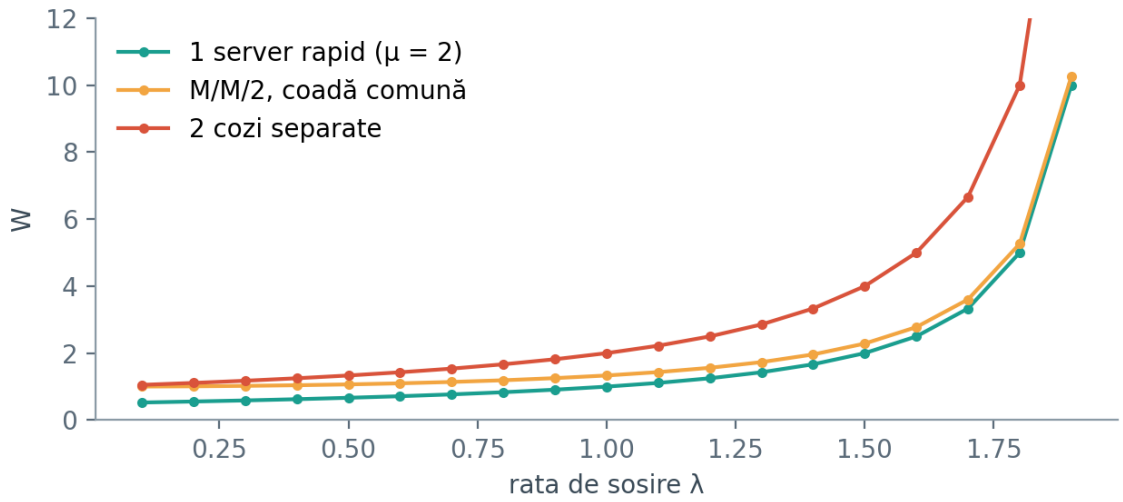


Figura 5. Timpul de răspuns pentru un server rapid, două servere cu coadă comună și două cozi separate, la aceeași capacitate totală

2.3. Capacitate finită: M/M/1/K și Erlang B

În M/M/1/K sistemul are loc pentru cel mult K clienți; cei în plus sunt refuzați (un buffer de router, o coadă limitată, HTTP 429). Distribuția este cea M/M/1 trunchiată: $\pi_k = \pi_0 \rho^k$ pentru $k \le K$, iar probabilitatea de blocare este $\pi(K) = (1-\rho)\rho^K/(1-\rho^{K+1})$; throughput-ul real este $\lambda' = \lambda(1-\pi(K))$. Pentru $\lambda = 10$, $\mu = 12$, $K = 15$ (Cambridge): $\pi(15) \approx 0{,}0114$, $\lambda' \approx 9{,}89$ clienți/s, utilizarea 0,824.

Când nu există deloc coadă (M/M/c/c), probabilitatea ca o cerere să fie pierdută este dată de **formula Erlang B**, $B(c, a) = (a^c/c!)/\Sigma_k\, a^k/k!$, calculată stabil prin recurența $B(0) = 1$, $B(c) = a\cdot B(c-1)/(c + a\cdot B(c-1))$. Pentru a = 10 erlangi, cu 10 resurse se pierd 21,5% dintre cereri, iar pentru sub 1% sunt necesare 18 resurse (Figura 6).

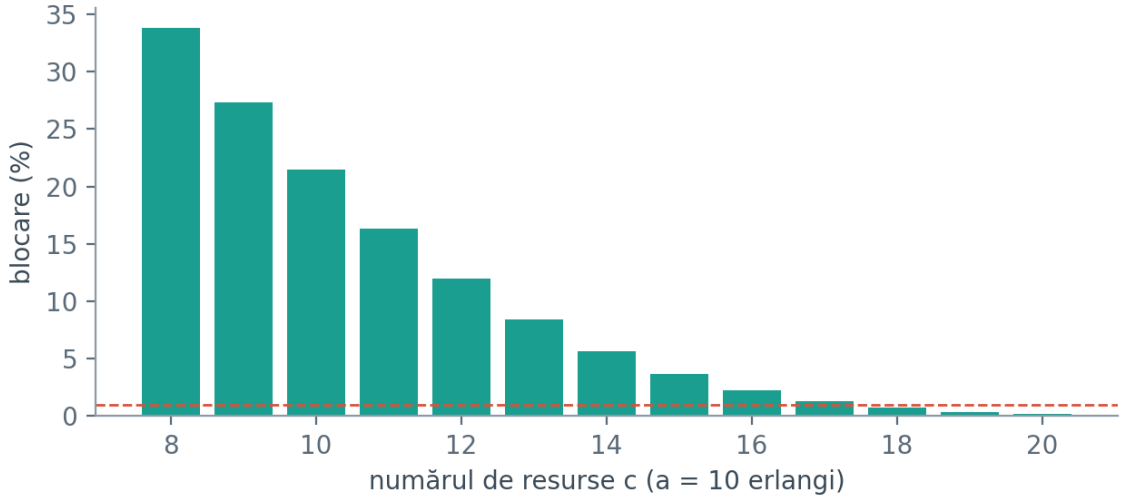


Figura 6. Probabilitatea de blocare Erlang B pentru a = 10 erlangi; linia punctată marchează 1%

2.4. Servire oarecare: Pollaczek–Khinchine și Kingman

Pentru sosiri Poisson și servire cu **orice** distribuție (media $1/\mu$, coeficientul de variație C_s), **formula Pollaczek–Khinchine** dă:

$$Wq = \rho\,(1 + C_s^{\,2})\,/\,(2\mu(1-\rho))$$

Servirea constantă ($C_s = 0$) înjumătățește așteptarea față de M/M/1, iar servirea foarte variabilă o mărește proporțional cu $1 + C_s^2$ (Figura 7). Simularea cu 400 000 de clienți la $\rho = 0{,}8$ a dat 1,96 (M/D/1), 3,89 (M/M/1) și 9,58 pentru o servire hiperexponențială cu $C_s^2 \approx 4$ (teoretic 2, 4 și 10). Pentru sosiri și serviri oarecare, **aproximarea lui Kingman** (1961) spune $Wq \approx V \cdot U \cdot T = ((C_a^2 + C_s^2)/2) \cdot (\rho/(1-\rho)) \cdot s$: trei pârghii – variabilitatea, utilizarea și timpul de servire.

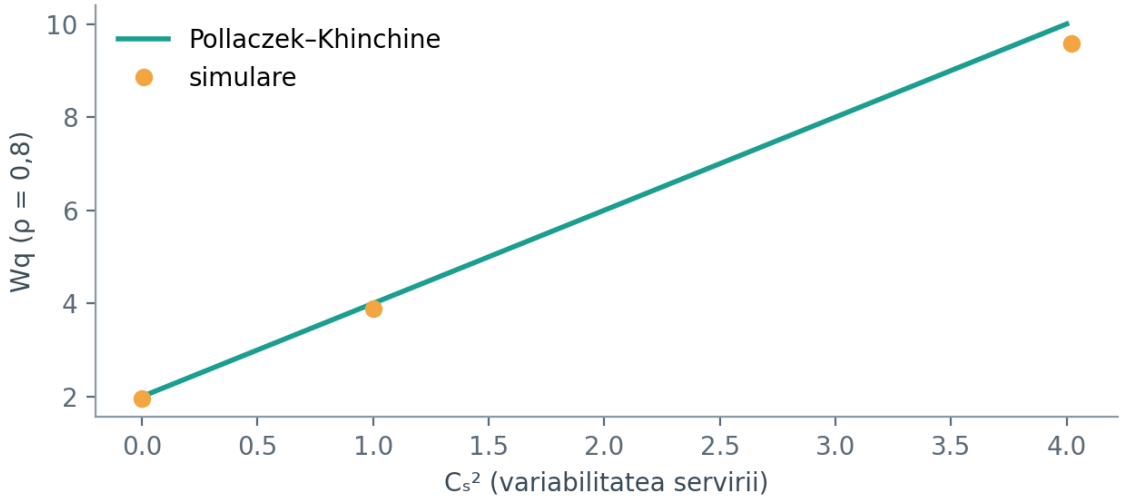


Figura 7. Așteptarea medie la $\rho = 0{,}8$ în funcție de variabilitatea servirii: formula Pollaczek–Khinchine și simulări

Formulele au limite: sosiri care nu sunt Poisson, cozi grele, priorități, abandon, blocări, percentile sau regim tranzitoriu. Pentru acestea folosim simularea cu evenimente discrete (săptămâna 5), validată pe cazurile cu formulă.

De reținut

M/M/1: $\pi_k = (1-\rho)\rho^k$, $P(N \geq n) = \rho^n$, $W = 1/(\mu - \lambda)$; latența explodează la $\rho \to 1$; $p99 \approx 4{,}6 \cdot W$. M/M/c: Erlang C, coada comună (pooling). M/M/1/K: blocare $\pi(K)$; M/M/c/c: Erlang B. M/G/1: $Wq \propto (1 + C_s^2)$; Kingman: $Wq \approx V \cdot U \cdot T$.

3. Rețele de cozi și sisteme reale

3.1. Rețele Jackson

O **rețea de cozi** are mai multe noduri, între care clienții se mută după probabilități de rutare q_{ij}. Într-o rețea **deschisă** clienții vin din exterior (rata γ_i) și pleacă; într-una **închisă** circulă un număr fix de clienți. Rata de sosire la fiecare nod se obține din **ecuațiile de trafic** $\lambda_i = \gamma_i + \Sigma_j \lambda_j q_{ji}$, un sistem liniar $(I - Q^T)\lambda = \gamma$. **Teorema lui Jackson** (1957): pentru sosiri externe Poisson, servire exponențială și rutare probabilistică, fiecare nod se comportă în regim staționar ca o coadă M/M/1 (sau M/M/c_i) independentă, cu rata λ_i.

Exemplu: o aplicație web cu $\gamma = 40$ de cereri/s, un server web ($\mu = 200$/s), un server de aplicație ($\mu = 120$/s) și o bază de date ($\mu = 150$/s); după aplicație, cu 0,5 cererea merge la baza de date și se întoarce, cu 0,5 pleacă. Ecuațiile de trafic dau $\lambda = (40, 80, 40)$: aplicația este vizitată de două ori. Utilizările sunt 0,2; 0,667 și 0,267, iar cu Little pe toată rețeaua $T = \Sigma L_i/\gamma = 65{,}3$ ms (o simulare cu evenimente discrete a dat 65,4 ms). Aplicația consumă 50 ms, adică peste trei sferturi din timp, și este gâtul de sticlă: sistemul se saturează la $\gamma = 60$/s

(Figura 8). Un cache care reduce apelurile aplicație → bază de date de la 0,5 la 0,25 scade λ(app) la ≈ 53/s și timpul de răspuns de la 65,3 la 28,7 ms, fără ca vreun server să devină mai rapid.

```
gamma = np.array([40.0, 0, 0])
Q = np.array([[0, 1.0, 0], [0, 0, 0.5], [0, 1.0, 0]])
mu = np.array([200.0, 120.0, 150.0])
lam = np.linalg.solve(np.eye(3) - Q.T, gamma)    # [40, 80, 40]
rho = lam / mu
T = (rho / (1 - rho)).sum() / gamma.sum()        # ≈ 0.0653 s
```

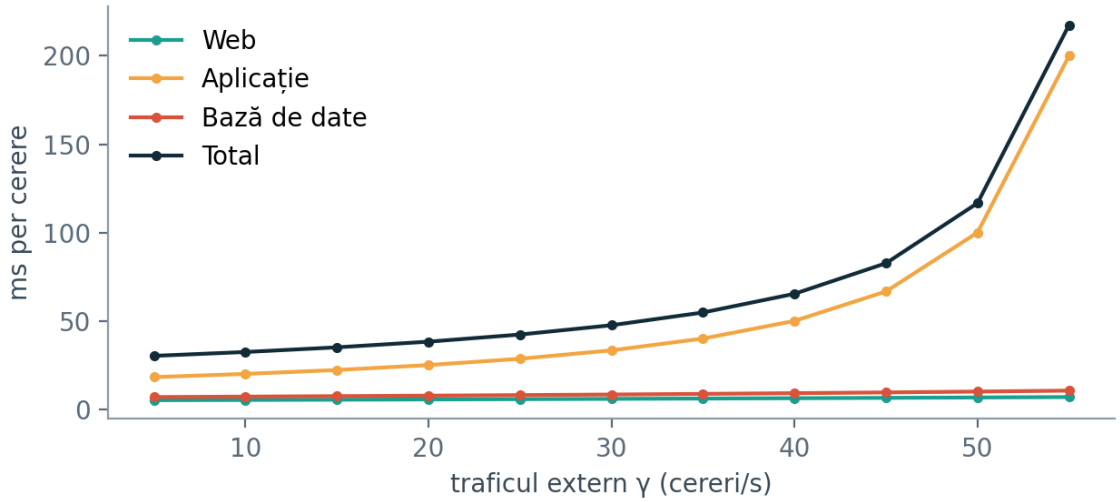


Figura 8. Contribuția fiecărui nod la timpul de răspuns, în funcție de traficul extern

Pentru rețele **închise** cu utilizatori interactivi, legea lui Little aplicată întregului ciclu dă **legea timpului de răspuns** R = N/X − Z (N utilizatori, throughput X, timp de gândire Z). Cu N = 100, Z = 5 s și X = 18 cereri/s, R ≈ 0,56 s. Rețelele închise mai mari se analizează cu analiza valorii medii (MVA).

3.2. Dimensionare, autoscaling, latența de coadă

Pentru λ = 90 de cereri/s și s = 100 ms (a = 9 erlangi), Erlang C dă: cu 10 servere P(așteptare) = 66,9% și W = 166,9 ms; cu 12 servere 26,6% și 108,9 ms; cu 14 servere 8,9% și 101,8 ms – randamente descrescătoare. **Regula rădăcinii pătrate** c ≈ a + β√a (aici 9 + 1·3 = 12) exprimă faptul că rezerva necesară crește doar ca √a, deci sistemele mari pot rula la utilizări mai mari. În autoscaling, ținta tipică de utilizare este 60–70%, pentru a lăsa rezervă pentru rafale și pentru curba latenței; pornirea instanțelor durează, de aceea se scalează după lungimea cozii sau latența p95, nu doar după CPU.

Latența de coadă la scară mare (Dean & Barroso, „The Tail at Scale”, 2013): dacă o cerere de utilizator apelează în paralel n servere, iar fiecare este lent în 1% din cazuri, fracțiunea cererilor lente este $1 - 0{,}99^n$: 9,6% pentru n = 10, 63,4% pentru n = 100. Percentilele rare ale unui server devin cazul obișnuit pentru utilizator; soluțiile includ cererile duplicate („hedged”), timeout-urile cu răspunsuri parțiale și reducerea fan-out-ului.

3.3. Servirea modelelor LLM

O cerere către un model mare de limbaj intră în coadă, așteaptă loc pe un GPU, trece prin **prefill** (procesarea promptului, până la primul token) și prin **decodare** (câte un token pe pas). Metricile sunt **TTFT** (timpul până la primul token = așteptare + prefill), **TPOT** (timpul pe token generat) și throughput-ul în tokeni/s. Legea lui Little arată memoria necesară: la 20 de cereri/s și 8 s pe generare, 160 de generări simultane ocupă memorie

GPU (cache-ul KV), deci serviciul este un sistem cu capacitate finită, cu control al admiterii (HTTP 429), limite de rată și priorități. **Batching-ul continuu** (Orca, 2022; vLLM cu PagedAttention, 2023) reface lotul la fiecare pas de decodare. Într-un model ilustrativ cu un pas de 20 ms + 0,5 ms × B, throughput-ul crește de la 49 tokeni/s (B = 1) la 1524 tokeni/s (B = 128), dar TPOT crește de la 20,5 la 84 ms – compromisul utilizare–latență din teoria cozilor.

Aceleași idei apar în cozile de mesaje (Kafka), pool-urile de conexiuni (M/M/c), limitarea de rată (token bucket), bufferele de rețea (bufferbloat), cozile de build-uri CI și platformele serverless (pornire la rece).

De reținut

Rețele Jackson: ecuații de trafic, fiecare nod = M/M/1, timpi adunați pe drumul cererii; gâtul de sticlă decide totul; reducerea vizitelor e adesea cea mai ieftină optimizare. Dimensionare: Erlang C, $c \approx a + \beta\sqrt{a}$, autoscaling la 60–70%. Tail at scale: $1 - 0{,}99^n$. LLM: TTFT, TPOT, batching continuu, Little pentru memorie.

Glosar RO – EN

Română	English
sistem de servire	service system
șir de așteptare, coadă	queue
stație de servire, server	server
disciplina cozii	queueing discipline
notația Kendall	Kendall notation
rată de sosire	arrival rate
rată de servire	service rate
grad de utilizare	utilisation
capacitate de trecere	throughput
timp de răspuns	response time
legea lui Little	Little's law
gât de sticlă	bottleneck
cerere de servire	service demand
proces naștere–moarte	birth–death process
încărcare oferită	offered load
probabilitate de blocare	blocking probability
coadă comună	shared queue (pooling)
capacitate finită	finite capacity

Română	English
coeficient de variație	coefficient of variation
rețea de cozi	queueing network
ecuații de trafic	traffic equations
probabilități de rutare	routing probabilities
dimensionare	capacity planning, staffing
latență de coadă	tail latency
timp până la primul token	time to first token (TTFT)
batching continuu	continuous batching
control al admiterii	admission control

Întrebări de autoevaluare

1. Ce înseamnă M/D/3/20 în notația Kendall?

a) 3 sosiri, 20 de servere

b) sosiri Poisson, servire constantă, 3 servere, cel mult 20 de clienți în sistem

c) servire exponențială, sosiri constante, 3 cozi

d) 20 de servere cu câte 3 locuri

2. Legea lui Little, $L = \lambda W$, este valabilă:

a) doar pentru M/M/1

b) doar pentru sosiri Poisson

c) pentru orice sistem stabil, indiferent de distribuții și disciplină

d) doar pentru servere cu disciplina FIFO

3. O cerere vizitează CPU-ul o dată (10 ms), discul de 3 ori (8 ms) și baza de date de 2 ori (15 ms). Care este gâtul de sticlă?

a) CPU-ul

b) discul

c) toate la fel

d) baza de date

4. Ce se întâmplă cu o coadă când $\rho = \lambda/\mu > 1$?

a) coada crește fără limită, aproximativ liniar în timp

b) coada se stabilizează la ρ clienți

c) throughput-ul devine λ

d) așteptarea devine zero

5. La aceeași utilizare $\rho = 0{,}8$, care sistem are așteptarea medie cea mai mică?

a) M/M/1

b) M/D/1

c) $M/H_2/1$ (servire foarte variabilă)

d) sunt egale

6. În M/M/1, timpul mediu de răspuns este:

a) $1/\mu$

b) λ/μ

c) $1/(\mu - \lambda)$

d) $\rho/(1 - \rho)$

7. Utilizarea unui server M/M/1 crește de la 0,8 la 0,9. Timpul mediu de răspuns:

a) crește cu 12,5%

b) crește cu 25%

c) rămâne aproape la fel

d) se dublează

8. Două servere identice, $\rho = 0{,}8$. Ce configurație are timpul mediu de răspuns mai mic?

a) două cozi separate

b) o coadă comună pentru ambele servere

c) sunt identice

d) depinde doar de disciplina FIFO

9. Formula Erlang B dă:

a) probabilitatea ca o cerere să fie pierdută când toate cele c resurse sunt ocupate (fără coadă)

b) timpul mediu de așteptare în M/M/c

c) numărul mediu de clienți în M/M/1

d) probabilitatea ca serverul să fie liber

10. În M/M/1/K, rata efectivă de sosire este:

a) λ

b) μ

c) $\lambda(1 - \pi(K))$

d) λ/K

11. Teorema lui Jackson spune că, într-o rețea deschisă de cozi exponențiale:

a) toate nodurile au aceeași utilizare

b) timpul de răspuns este cel al celui mai lent nod

c) fluxurile interne sunt întotdeauna Poisson

d) fiecare nod se comportă ca o coadă M/M/1 independentă, cu rata din ecuațiile de trafic

12. De ce se dimensionează serviciile interactive pentru o utilizare de 60–70%, nu de 95%?

a) pentru că latența crește ca $1/(1 - \rho)$ și explodează aproape de 1

b) pentru că serverele se defectează la 95%

c) pentru că throughput-ul scade peste 70%

d) pentru că legea lui Little nu mai e valabilă

13. O cerere de utilizator apelează în paralel 50 de servere, fiecare lent în 1% din cazuri. Aproximativ ce fracțiune din cereri sunt lente?

a) 1%

b) 5%

c) 40%

d) 50%

14. În servirea unui LLM, TTFT (time to first token) include:

a) doar timpul de decodare

b) așteptarea în coadă și procesarea promptului (prefill)

c) timpul total al răspunsului

d) timpul dintre două tokenuri

15. După aproximarea lui Kingman, $Wq \approx V \cdot U \cdot T$. Care dintre acțiuni reduce factorul V?

a) adăugarea de servere

b) un procesor mai rapid

c) mai multă memorie

d) uniformizarea cererilor și netezirea rafalelor

16. Un API primește în medie 50 de cereri/s, iar timpul mediu de răspuns este 200 ms. Câte cereri sunt în curs de procesare, în medie?

(Răspuns numeric)

17. La un server sosesc 30 de cereri/min, iar servirea durează în medie 1,5 s. Care este gradul de utilizare ρ?

(Răspuns numeric)

18. O coadă M/M/1 are $\lambda = 8$ cereri/s și $\mu = 10$ cereri/s. Care este timpul mediu de răspuns W, în secunde?

(Răspuns numeric)

19. O coadă M/M/1 are $\rho = 0{,}5$. Care este probabilitatea să fie cel puțin 3 clienți în sistem?

(Răspuns numeric)

20. Traficul oferit este $a = 16$ erlangi. Câte servere alegem după regula rădăcinii pătrate cu $\beta = 1$?

(Răspuns numeric)

Răspunsuri

1. b) – A/B/c/K: sosiri M (exponențiale), servire D (constantă), c = 3 servere, capacitate K = 20.

2. c) – Little nu face nicio ipoteză despre distribuții, disciplină sau numărul de servere.

3. d) – Cererile de servire: 10, 24 și 30 ms; maximul (30 ms) este la baza de date.

4. a) – Clienții sosesc mai repede decât pot fi serviți; surplusul $\lambda - \mu$ se acumulează.

5. b) – Pollaczek–Khinchine: $Wq \propto (1 + C_s^2)$; servirea constantă are $C_s = 0$.

6. c) – $W = 1/(\mu - \lambda)$; $\rho/(1 - \rho)$ este L, iar $1/\mu$ este doar timpul de servire.

7. d) – $W/s = 1/(1 - \rho)$: de la 5 la 10.

8. b) – Coada comună (pooling) evită situația în care un client așteaptă în timp ce alt server e liber: W scade de la 5 la 2,78.

9. a) – Erlang B este probabilitatea de blocare într-un sistem M/M/c/c, fără coadă.

10. c) – Fracțiunea $\pi(K)$ a clienților găsește sistemul plin și este refuzată.

11. d) – Rețeaua se descompune în noduri independente; timpii se adună pe drumul cererii.

12. a) – La $\rho = 0{,}65$, $W \approx 3s$; la $\rho = 0{,}95$, $W = 20s$.

13. c) – $1 - 0{,}99^{50} \approx 0{,}39$.

14. b) – TTFT = așteptare + prefill; timpul dintre tokenuri este TPOT.

15. d) – V măsoară variabilitatea sosirilor și a servirii; U depinde de capacitate, T de viteza servirii.

16. 10 – $L = \lambda W = 50 \cdot 0{,}2 = 10$.

17. 0,75 – $\lambda = 0{,}5/s$; $\rho = \lambda \cdot s = 0{,}5 \cdot 1{,}5 = 0{,}75$.

18. 0,5 – $W = 1/(\mu - \lambda) = 1/2 = 0{,}5$ s.

19. 0,125 – $P(N \geq 3) = \rho^3 = 0{,}125$.

20. 20 – $c = a + \beta\sqrt{a} = 16 + 4 = 20$.

Bibliografie

- Ș. Holban – Tehnici de modelare și simulare, cap. 2 (modelarea cu graful activităților) și cap. 3 (sisteme cu șiruri de așteptare) (materiale de curs).
- R. J. Gibbens – Computer Systems Modelling, University of Cambridge – legea lui Little, M/M/1, M/M/c, M/M/1/K, M/M/c/c, M/G/1, rețele Jackson.
- M. Harchol-Balter – Performance Modeling and Design of Computer Systems: Queueing Theory in Action, Cambridge University Press, 2013.
- L. Kleinrock – Queueing Systems, Volume 1: Theory, Wiley, 1975.
- R. Jain – The Art of Computer Systems Performance Analysis, Wiley, 1991.
- E. D. Lazowska, J. Zahorjan, G. S. Graham, K. C. Sevcik – Quantitative System Performance, Prentice Hall, 1984.
- J. D. C. Little – A Proof for the Queuing Formula: $L = \lambda W$, Operations Research, 9(3), 1961.
- J. F. C. Kingman – The single server queue in heavy traffic, Proceedings of the Cambridge Philosophical Society, 1961.
- J. Dean, L. A. Barroso – The Tail at Scale, Communications of the ACM, 56(2), 2013.
- G.-I. Yu et al. – Orca: A Distributed Serving System for Transformer-Based Generative Models, OSDI 2022.

- W. Kwon et al. – Efficient Memory Management for Large Language Model Serving with PagedAttention, SOSP 2023.
- Material realizat cu ajutorul Claude Fable 5.1 și Claude Opus 5.5.